

LABSHEET 9

- Step 1. Install and import the required libraries, including OpenCV and the chosen deep learning framework (e.g., Ultralytics YOLO or TensorFlow).

```
!pip install ultralytics -q
```

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
```

```
print("Libraries imported successfully.")
```

```
print("Name: Aryan Agarwal")
print("CU240251800")
```

```

_____ 46.6/46.6 kB 660.7 kB/s eta 0:00:00
_____ 1.4/1.4 MB 7.0 MB/s eta 0:00:00
_____ 53.2/53.2 kB 3.1 MB/s eta 0:00:00
_____ 77.7/77.7 kB 3.7 MB/s eta 0:00:00

```

```
Creating new Ultralytics Settings v0.0.8 file 
```

```
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
```

```
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com
```

```
Libraries imported successfully.
```

```
Name: Aryan Agarwal
CU240251800
```

- Step 2. Load a pre-trained object detection model such as YOLOv8, SSD, or Faster R-CNN

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n.pt")
```

```
print("YOLOv8 pre-trained model loaded successfully.")
```

```
print("Name: Aryan Agarwal")
print("CU240251800")
```

```

Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% _____ 6.2MB 2
YOLOv8 pre-trained model loaded successfully.

```

```
Name: Aryan Agarwal
CU240251800
```

- Step 3. Download or access images from a standard dataset such as MS COCO, Pascal VOC, or Open Images Dataset.

```

import cv2
import matplotlib.pyplot as plt

url = "https://ultralytics.com/images/bus.jpg"

import requests
data = requests.get(url).content

with open("bus.jpg", "wb") as f:
    f.write(data)

img = cv2.imread("bus.jpg")

plt.figure(figsize=(4, 3))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Input Image")
plt.axis("off")
plt.show()
plt.close()

```

```
print("Image loaded successfully.")  
  
print("Name: Aryan Agarwal")  
print("CU240251800")
```

Input Image



Image loaded successfully.
Name: Aryan Agarwal
CU240251800

Step 4. Perform any necessary image preprocessing, including resizing and normalization, before inference.

```
img = cv2.resize(img, (640, 480))  
  
plt.figure(figsize=(4, 3))  
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))  
plt.title("Preprocessed Image")  
plt.axis("off")  
plt.show()  
plt.close()  
  
print("Image resized successfully.")  
  
print("Name: Aryan Agarwal")  
print("CU240251800")
```

Preprocessed Image



Image resized successfully.
Name: Aryan Agarwal
CU240251800

Step 5. Apply the object detection model to identify and localize objects within the selected images.

```
results = model(img)  
  
print("Object detection completed.")  
  
print("Name: Aryan Agarwal")  
print("CU240251800")
```

```
0: 480x640 4 persons, 1 bus, 1 truck, 495.6ms
Speed: 18.5ms preprocess, 495.6ms inference, 56.9ms postprocess per image at shape (1, 3, 480, 640)
Object detection completed.
Name: Aryan Agarwal
CU240251800
```

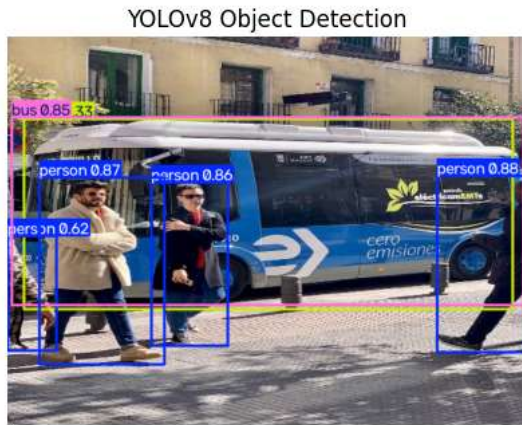
- ✓ Step 6. Display the detected objects with bounding boxes, class labels, and confidence scores.

```
result_img = results[0].plot()

plt.figure(figsize=(5, 4))
plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB))
plt.title("YOLOv8 Object Detection")
plt.axis("off")
plt.show()
plt.close()

print("Bounding boxes, labels and confidence scores displayed.")

print("Name: Aryan Agarwal")
print("CU240251800")
```



Bounding boxes, labels and confidence scores displayed.
Name: Aryan Agarwal
CU240251800

- ✓ Step 7. Evaluate the detection results by analyzing correctly detected objects, false detections, and missed objects.

```
for box in results[0].boxes:
    cls = int(box.cls[0])
    conf = float(box.conf[0])
    print(model.names[cls], "Confidence:", round(conf, 2))

print("Higher confidence indicates more reliable detections.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

```
person Confidence: 0.88
person Confidence: 0.87
person Confidence: 0.86
bus Confidence: 0.85
person Confidence: 0.62
truck Confidence: 0.33
Higher confidence indicates more reliable detections.
Name: Aryan Agarwal
CU240251800
```

- Step 8. Compare the performance of different object detection models in terms of detection accuracy and inference speed (if multiple models are used).

Ans. YOLO is designed for fast real-time detection. SSD also provides fast detection with

- ✓ moderate accuracy, while Faster R-CNN generally provides high detection accuracy but requires more computation.

```
print("YOLOv8 : Fast and suitable for real-time detection")
print("SSD : Fast with moderate-to-high accuracy")
print("Faster R-CNN : High accuracy but slower")
```

```
print("Name: Aryan Agarwal")
print("CU240251800")
```

```
YOLOv8 : Fast and suitable for real-time detection
SSD : Fast with moderate-to-high accuracy
Faster R-CNN : High accuracy but slower
Name: Aryan Agarwal
CU240251800
```

Step 9. Analyze the suitability of the selected model for real-world applications such as autonomous driving, surveillance, retail analytics, or industrial inspection

Ans. YOLOv8 is suitable for real-time applications because it can detect multiple objects quickly. It can be used in surveillance, traffic monitoring, retail analytics, and autonomous systems.

```
print("YOLOv8 is suitable for real-time object detection.")
print("Applications: surveillance, traffic monitoring, retail and autonomous systems.")
```

```
print("Name: Aryan Agarwal")
print("CU240251800")
```

```
YOLOv8 is suitable for real-time object detection.
Applications: surveillance, traffic monitoring, retail and autonomous systems.
Name: Aryan Agarwal
CU240251800
```

Step 10. Summarize the observations and discuss the advantages and limitations of deep learning-based object detection techniques.

Ans. The experiment successfully detected multiple objects using YOLOv8 with bounding boxes, class labels, and confidence scores.

Advantages:

High detection accuracy, Detects multiple objects simultaneously, Works with complex images, Suitable for real-time applications.

Limitations:

Requires considerable computational resources, Performance depends on training data, Small or heavily occluded objects can be difficult to detect.

```
print("Observation:")
print("YOLOv8 successfully detected and localized objects.")
print("Advantages: Fast, accurate and supports multiple objects.")
print("Limitations: Requires computational resources and may miss small or occluded objects.")
```

```
print("Name: Aryan Agarwal")
print("CU240251800")
```

```
Observation:
YOLOv8 successfully detected and localized objects.
Advantages: Fast, accurate and supports multiple objects.
Limitations: Requires computational resources and may miss small or occluded objects.
```

Name: Aryan Agarwal
CU240251800

Questions:

- Q1. Differentiate between image classification, object detection, and image segmentation.

Task Description

Image classification assigns a class or category to the entire image. Object detection identifies multiple objects in an image and locates them using bounding boxes along with their class labels. Image segmentation goes a step further by classifying individual pixels of the image, allowing the exact shape and boundaries of objects to be identified.

- Q2. Explain the working principle of the YOLO (You Only Look Once) object detection algorithm.

YOLO divides an image into a grid and processes the entire image in a single neural-network pass. It predicts bounding boxes, class labels, and confidence scores simultaneously. Non-Maximum Suppression (NMS) is then used to remove duplicate bounding boxes.

- Q3. Compare YOLO, SSD, and Faster R-CNN based on speed, accuracy, and practical applications.

YOLO is generally designed for very fast, real-time object detection while maintaining good accuracy, making it suitable for applications such as surveillance, traffic monitoring, and autonomous systems. SSD also provides fast detection with good accuracy and is commonly used in real-time and resource-constrained applications. Faster R-CNN generally focuses more on detection accuracy than speed and requires more computational resources, making it suitable for applications where accurate object localization is more important than real-time performance.

- Q4. What is a bounding box? Why is it important in object detection?

A bounding box is a rectangular box drawn around a detected object. It specifies the object's location and size in an image and helps the system identify where each object is present.

- Q5. Explain the significance of confidence score and Intersection over Union (IoU) in object detection.

A confidence score indicates how certain the model is that a detected object belongs to a particular class. IoU measures the overlap between the predicted bounding box and the actual bounding box. Higher IoU indicates better localization.

- Q6. What are the advantages of using pre-trained deep learning models over traditional image processing techniques?

Pre-trained models can detect complex objects without manually designing features. They provide better accuracy, require less training data and development time, and can be fine-tuned for specific applications.

- Q7. Why are datasets such as MS COCO and Pascal VOC widely used for object detection research?

MS COCO and Pascal VOC contain large collections of images with annotated objects and bounding boxes. They provide standard benchmarks for training, testing, and comparing different object detection models.

✓ Q8. Mention five real-world applications of object detection in computer vision.

1. Autonomous vehicles
2. Surveillance and security
3. Medical image analysis
4. Retail and inventory management
5. Traffic monitoring

✓ Q9. What challenges are commonly encountered while detecting objects in complex realworld environments?

Common challenges include changes in lighting, object occlusion, different object sizes, complex backgrounds, camera movement, low image quality, and objects appearing at different orientations.

✓ Q10. How can object detection systems be further improved using recent advancements in deep learning and computer vision?

Object detection can be improved using advanced architectures, better training datasets, data augmentation, transfer learning, attention mechanisms, improved feature extraction, and more efficient models for real-time detection.